

task-spec

SKILLS/TASK-SPEC · ENGINEERING REFERENCE · SKILL V3.2.0 ·
FORMAT V3

The unit of work that *proves itself*.

Most AI-coding "tasks" are PRDs — written for humans, judged by humans, which keeps a human stuck in the middle of every agent loop. A **Task-Spec carries its own runnable bash success criteria**: the agent reads the spec, does the work, runs the evals, and loops on failure without supervision. Humans sit only at the edges. This is **Eval-Driven Development**.

9

WORKFLOW PHASES

6

ZONES PER SPEC

20

BUNDLED SCRIPTS

2

GATES — DUALS

DOCUMENT

Skill anatomy & format reference · v3.2.0

DONE MEANS

accept-task.sh wrote accepted: true

The product is a file: tasks/T-YYYYMMDD-<slug>.md — YAML frontmatter + six zones + ≥ 3 runnable bash evals + an Exit Check. One format, many executors: Claude, Codex, Kimi, Cursor, taskship, anthive, a plain bash loop, or a human with a terminal.

Introduction

Why task-spec exists

Every task format before this one was a promise. A ticket, a PRD, a checklist — all of them describe work in prose and leave the judgment of "done" to a human reader. That shape breaks the moment an autonomous agent enters the loop: the agent finishes, declares victory, and someone has to check. The human is back in the middle, supervising every iteration — which is exactly the bottleneck agents were meant to remove.

The main idea — the eval defines done

A Task-Spec inverts the contract. The definition of done travels **inside the task file** as executable bash: at least three `eval_N()` functions plus an Exit Check. The executor implements, runs the evals, and loops on failure within a declared budget — no human in the middle. Humans hold exactly two positions: **intent-setting** (Phase 1) and **acceptance review** (Phase 9). Everything between the gates runs without one.

One format, many executors

The spec is deliberately vendor-neutral. `execution_backend` is an open string, not an enum; seven dispatch recipes ship in the box (claude-code, codex, kimi, gemini, taskship, anthive, custom), and a conformance suite makes "any conformant engine can pick it up" a **testable claim**, not marketing.

ATOMIC

One unit

one indivisible piece of work per file — the smallest thing that can be built and verified on its own.

SELF-VERIFYING

Carries its evals

≥3 runnable bash evals + an Exit Check live inside the spec — the definition of done is executable.

VENDOR-NEUTRAL

Any engine

Claude, Codex, Kimi, Cursor, taskship, anthive, or a human — one format, many executors, certified L0–L2.

THE INVARIANT

A task is provably DONE only when `accept-task.sh` writes `accepted: true` — after the evals pass from a clean checkout, the change set stayed inside the blast radius, and the sign-off HMAC is intact. Done is stamped by a gate, never asserted by an executor.

Contents

—	Introduction	why the eval defines done
1	The Closed Loop	author → gate → execute → accept
2	The Nine Phases	understand → accept, one by one
3	The Two Gates Are Duals	PRE certifies the spec · POST certifies the work
4	Anatomy of a T-*.md	frontmatter + six zones · Zone 3 is the moat
5	The CAW Triad	Capability · Agent · Workers
6	Folder Map	what lives where, and why
7	The Scripts, By Role	20 bash CLIs — the deterministic machine
8	The Effort Gate	size-tiered, engine-aware routing
9	Executor Conformance	LO–L2 · A2A interop
10	Backlog Architecture	five layers of state · the thesis

1 The closed loop.

AUTHOR → GATE → EXECUTE → ACCEPT · *the full v3 lifecycle*

The loop is **closed**: Phase 7 certifies the **spec** is delegate-safe (evals are well-formed bash; assertion failure is expected — the work isn't built yet), and Phase 9 certifies the **work** is real (evals now pass from a clean checkout, the change set stayed inside the blast radius, the sign-off HMAC is intact).



FIG 1 The closed loop – seven authoring phases produce a stamped spec, the executor runs unsupervised between the gates, and only the POST-gate can mint done. The effort gate refuses XL and subjective work outright, routing it to spec-driven development.

2

The nine phases, one by one.

PHASES 1–7 AUTHOR · 8 HANDS OFF · 9 ACCEPTS

#	PHASE	TOOL	WHAT HAPPENS
1	Understand	—	Parse intent into five answers: description, effort class (XS/S/M/L/XL), agent hint, provenance, best-guess <code>touches_paths</code> . If vague — ask once, never proceed fuzzy.
2	Research	Context7 · Exa · Ref	MCP validation of the domain: official docs, production examples, canonical references. Findings feed Anti-Patterns and Do-Not-Touch with real failure modes.
3	Scan	—	Read-only pass over the host repo: do the paths exist, do in-flight tasks collide, what conventions (CLAUDE.md, editorconfig) apply.
4	Architect	task-architect	The judgment layer. Applies the Agreement Matrix (KB + MCP), picks isolation, drafts the 3+ evals, returns a build plan with a confidence score. Below the severity threshold (0.80–0.99) it asks the user before composing.
5	Compose	generate-task-spec.sh	Scaffold <code>T-*.md</code> from the template, fill frontmatter + all six zones. New specs are <code>format_version: 3</code> ; legacy v0/v1/v2 validate with warnings.
6	Validate	validate-task-spec.sh	The linter, not the gate. Structural checks only: zones present, no TODO stubs, evals are valid bash, deps exist, Exit Check calls every eval, behavior↔eval traceability.
7	Gate · PRE	safe-to-delegate.sh - -stamp	The only path to <code>signed_off: true</code>. Validate + eval execution → one go/no-go verdict. Assertion failure is expected; bash-level errors are not. Stamps an HMAC-SHA256 envelope — hand-stamping is rejected.
8	Dispatch	dispatch-recipes/	Hand the stamped spec to the executor named by <code>execution_backend</code> — an open string, not an enum. Seven recipes bundled.
9	Accept · POST	accept-task.sh -- stamp	The opposite question: is the work real? Re-runs evals from a clean checkout (must PASS), checks blast radius, re-verifies the HMAC, optional <code>--gold-sanity</code> Goodhart guard. Writes <code>accepted: true</code> .

FAST BATCH

A shortcut exists for known domains: skip Phases 2 and 4 and bulk-generate stubs with `batch-generate.sh` from a `slug: description` intent file, then bulk-validate. The gates are never skipped — only the research and judgment phases collapse.

3

The two gates are duals.

SAME EVALS · OPPOSITE EXPECTATIONS

Before the work, the evals **should fail** — for the right reason. After the work, the evals **must pass** — from a clean checkout. Everything between the gates runs without a human.

PRE-GATE · PHASE 7

"Are the evals well-formed enough to delegate?"

- structural validation must exit 0
- eval bodies execute with **no bash-level errors** — no syntax errors, unbound variables, missing commands, inverted-grep footguns
- assertion failure is **EXPECTED** — the work isn't built yet; the gate fails only for the wrong reason (broken bash)
- verdict: **DELEGATE** or **DO NOT DELEGATE**
- stamps **signed_off: true** + **signed_off_sig** (HMAC-SHA256)

POST-GATE · PHASE 9

"Is the work REAL?"

- evals re-run from a **clean checkout** — now they must PASS
- **blast radius**: change set stays inside `touches_paths`, outside do-not-touch
- **HMAC re-verified**: eval bodies unchanged since sign-off — nobody weakened the tests to pass them
- **--gold-sanity**: Goodhart guard — blocks an eval that also passes on the unpatched baseline
- stamps **accepted: true** — closing the loop

THE HONEST TRUST CLAIM

The HMAC envelope is **tamper-evident, not tamper-proof**. It proves the evals weren't edited after stamping (repo-shared key, provisioned by `setup-taskspec-signing-key.sh`). It is a drift and accidental-edit guard, not a security boundary. The claim is "humans at intent + review" — never "no humans."

WHY DUALITY MATTERS

A single gate can be gamed from either side: a spec with broken evals sails through execution, or an executor weakens the tests to pass them. Two gates holding the same evals to opposite expectations close both holes — the PRE-gate proves the contract is executable, the POST-gate proves it was honoured.

4

Anatomy of a T-*.md.

YAML FRONTMATTER + SIX ZONES · *Zone 3 is the moat*

```
tasks/T-20260707-add-health-endpoint.md

--- YAML FRONTMATTER ----- machine-parseable ---
id · title · status · format_version: 3 · profile · effort
budget_iterations · agent · depends_on · touches_paths · source_note
signed_off: false      ← only the PRE-gate (Phase 7) flips this
accepted:  false      ← only the POST-gate (Phase 9) flips this

## 1 · INTENT           why + goal + lean context (≤100 lines – link, don't duplicate)
## 2 · BEHAVIOR         B-1, B-2 ... Given / When / Then           [standard + full]
## 3 · CONTRACT ♦ THE MOAT – the only executable task format
  Success Criteria      ≥3 bash eval_N() – terminal, idempotent, cheap → expensive
  Validation Card       YAML mirror: retry policy · agent contract · timeouts
  Exit Check            eval_1 && eval_2 && eval_3

## 4 · GUARDRAILS      anti-patterns with reasons + do-not-touch paths
## 5 · OPERATIONS      open questions – unresolved ⇒ status: blocked
## 6 · ROLLBACK + OBSERVABILITY                                [full profile only]
```

FIG 2 The v3 file shape – every other task format is readable; this one is executable. The two stamped fields in frontmatter can only be flipped by their gates.

ZONE	WHAT IT HOLDS – AND THE RULE IT ENFORCES
1 · Intent	One-line Why + Goal + lean Context. If Zone 1 outweighs the Contract, you wrote a PRD, not a Task-Spec.
2 · Behavior	<code>B-N</code> clauses in plain language. Every behavior needs ≥1 eval; every <code>verifies: [B-N]</code> points at a declared behavior — no orphans either way.
3 · Contract ♦	The moat. Success Criteria + Validation Card + Exit Check. Evals are ordered cheap→expensive so failures surface fast.
4 · Guardrails	Specific don'ts with reasons, mined from MCP research, + exact do-not-touch paths. "Be careful" is rejected; if empty, write <code>(none)</code> explicitly.
5 · Operations	Admitted unknowns. An unresolved question is a first-class blocker — it sets <code>status: blocked</code> (A2A <code>input-required</code>) so the spec never reaches the gate until answered.
6 · Rollback + Obs.	Revert steps + expected duration, metrics, alert conditions. Required for unattended dispatch or security / financial-critical severity.

PROFILES SCALE THE REQUIREMENT

Which zones are required scales with `profile` (`lite` / `standard` / `full`). The irreducible core — Goal, runnable Success Criteria, Exit Check — is present at every profile; Rollback + Observability only on `full`.

5

The CAW Triad — Capability, Agent, Workers.

DETERMINISM · JUDGMENT · FRESH KNOWLEDGE — IN SEPARATE PLACES

The skill is the "cornerstone CAW": a three-layer bundle where the deterministic workflow, the judgment, and the fresh knowledge live in separate places — so each can be trusted, tested, and replaced on its own terms.

C · CAPABILITY

The Skill

`SKILL.md` + 20 bash scripts + the template. The deterministic layer: workflow phases, linter, gates, state management. Portable as a single folder — copy it, run `install.sh`, and `/task-spec` works in every repo.

A · AGENT

task-architect

The judgment layer, spawned in Phase 4 with isolated context. Classifies effort, applies the Agreement Matrix (KB + MCP), drafts evals and guardrails, returns a build plan with a confidence score gated at 0.80–0.99 by severity.

W · WORKERS

The MCPs

Fresh validation at authoring time: **Context7** (official docs), **Exa** (production examples), **Ref** (canonical references). Their findings ground the guardrails in real failure modes, not guesses.

WHY THE SEPARATION MATTERS

Everything *enforceable* is a bash CLI, not prose — the core gate path (`validate`, `safe-to-delegate`, `run-task-spec`, `_lib.sh`) runs on macOS system bash 3.2 with zero setup.

Everything *judgmental* is one agent with an inspectable confidence score. Everything *perishable* (library knowledge) is fetched live at authoring time. Determinism never depends on a model; judgment never hides in a script.

THE JUDGMENT STAYS VISIBLE

The task-architect never silently proceeds under its confidence threshold — it asks the user once, then composes. Confidence is arithmetic over evidence (KB agreement × MCP agreement × modifiers), printed in the build plan — auditable, not vibes-based.

6 Folder map — what lives where, and why.

ONE PORTABLE DIRECTORY · THE WHOLE TRIAD TRAVELS TOGETHER

```
task-spec/
├─ SKILL.md                - the contract Claude loads; 9-phase workflow + CLI index
├─ README.md              - human pitch, quick start, worked example
├─ CHANGELOG.md           - format + skill version history
├─ plugin.json · marketplace.json - packaging + listing metadata
├─ agents/                - task-architect.md, the A of the triad
├─ configs/               - setup-taskspec-signing-key.sh (HMAC envelope key)
├─ evals/                 - evals.json: 6 skill-trigger benchmark cases
├─ references/            - the knowledge base the agent navigates
│  └─ concepts/           - 11 definitional docs (≤150 lines each)
│  └─ patterns/           - 5 implementation how-tos (≤200 lines each)
│  └─ schemas/            - published JSON Schemas (Draft 2020-12)
│  └─ examples/           - Python + TypeScript reference consumers
│  └─ index.md · quick-reference.md - navigation + one-page cheatsheet
├─ runbooks/              - 14 operational playbooks (workflows, recovery)
│  └─ dispatch-recipes/   - 7 per-engine adapters (non-normative layer)
├─ scripts/               - 20 bash CLIs; core gate path is bash-3.2-safe
├─ templates/             - task-spec.md.tpl, the T-*.md scaffold
├─ tests/                 - self-contained suite (tempdir, no real backlog)
│  └─ conformance/        - 6 T-conformance-*.md + run_conformance.sh driver
│  └─ fixtures/           - 17 adversarial specs + oracle.json verdicts
```

FIG 3 The skill directory – the C, A, and W of the triad ship as one folder.

FOLDER	INTENT
references/	<i>Concepts</i> answer "what IS X?" (format spec, EDD, zones, profiles, conformance, signed-off); <i>patterns</i> answer "how do I write X?" (idempotent evals, YAML cards); <i>schemas</i> make the format machine-consumable; <i>examples</i> prove cross-language consumption.
runbooks/	Entry paths (fuzzy intent, meeting note, legacy task, batch sprint, decomposition), lifecycle ops (validating, dispatching, crash recovery), and the dispatch-recipes adapter layer — one per engine, same five-section shape.
tests/	Six suites (author-flow e2e, HMAC envelope, extractor fuzz, bash-3.2 portability, cross-engine equivalence, v3 closed loop) + the executor-conformance suite + 17 adversarial fixtures with an oracle of expected verdicts.

7

The scripts, grouped by role.

THE THREE-COMMAND AUTHOR FLOW: GENERATE → VALIDATE → SAFE-TO-DELEGATE --STAMP

STAGE	SCRIPT	PURPOSE
author	<code>generate-task-spec.sh</code>	Scaffold one T-*.md from the template.
author	<code>batch-generate.sh</code>	Fast Batch: N stubs from a <code>slug: description</code> intent file, then bulk-validate.
author	<code>migrate-legacy-task.sh</code>	Convert a legacy markdown checklist into runnable eval stubs + validation card.
gate	<code>validate-task-spec.sh</code>	Pre-gate structural linter (17+ checks). Opt-in: <code>--shellcheck-evals</code> , <code>--dry-run-eval</code> ; <code>--emit-schema</code> prints the JSON Schema.
gate	<code>run-task-spec.sh</code>	Execute each eval_N in an isolated subshell + the Exit Check; <code>--ci</code> emits JSON per eval.
gate	<code>safe-to-delegate.sh</code>	THE PRE-gate: validate + eval execution → DELEGATE / DO-NOT-DELEGATE; <code>--stamp</code> writes signed_off + HMAC.
accept	<code>accept-task.sh</code>	THE POST-gate: clean-checkout eval pass + blast radius + HMAC recheck → accepted: true; <code>--gold-sanity</code> adds the Goodhart baseline guard.
backlog	<code>transition-status.sh</code>	Atomic status change with lock + ledger — never hand-edit frontmatter status.
backlog	<code>list-ready · archive · backup-backlog</code>	Queue view, done/parked archival, tarball snapshots.
backlog	<code>lint-backlog · query-metrics</code>	Cross-task linter (path overlaps, dependency cycles, duplicate ids) + metrics ledger queries. Require bash 4+; self-guard and re-exec.
state	<code>rebuild-state · install-hooks</code>	Regenerate <code>_state.yaml</code> from frontmatter (crash recovery); pre-commit hook to catch drift.
conform	<code>conformance-check · ref-executor</code>	Certify an executor at L0/L1/L2; ref-executor is the ~60-line worked example of the consumer contract.
install	<code>install · lint-skill-docs · _lib.sh</code>	Agent install, docs lint, and the shared bash library (version source of truth, A2A mapping helpers).

THE RULE

Everything enforceable is a bash CLI, not prose. The core gate path runs on macOS system bash 3.2 with zero setup — determinism must not depend on anyone's environment.



The effort gate — size-tiered, engine-aware routing.

NEW IN v3.2 · WHEN IN DOUBT, CLASSIFY UP

The size-based safety primitive. Small work is atomic and portable; L is the boundary case one long-horizon engine can hold as a single coherent spec; XL routes out to spec-driven development entirely. The gate sits immediately after Phase 1 — misclassified scope is refused before a single zone is written.

CLASS	TIME SCOPE	SIGNAL	VERDICT	ENGINE
XS	≤ 2 hours	Trivial one-liner, config tweak	Task-Spec	Kimi (<i>sprinter</i>)
S	≤ 1 day	One file or 2–3 closely-related files	Task-Spec	Kimi
M	1–3 days	Module-level change, coordinated files	Task-Spec	Kimi
L	3–7 days	Major migration held as ONE coherent goal — requires <code>execution_backend: glm</code> ; any other backend at L is refused	Conditional	GLM (<i>marathoner</i>)
XL	> 1 week	Multi-team, platform rewrite; also anything subjective (UX, copy, design)	Refused	SDD — AgentSpec · OpenSpec · SpecKit

THE REFUSALS ARE CRITICAL-CLASS

XL is never overridden — decompose into S/M atoms or route to SDD. Subjective output always routes to SDD: bash can't check beauty. A task that feels L because it's two M tasks bundled should be **decomposed** — decomposition restores eligibility, and the flat parent-index + detail-atoms shape (with `depends_on` edges) is the converged pattern.

WHY THE GATE EXISTS

An eval-gated format is only honest at scopes an eval can actually cover. Small units get real, cheap, deterministic evals; a platform rewrite does not. The effort gate is the format admitting its own boundary — the same intellectual honesty as Converge's "green evals, not correct outcomes."

9 Executor conformance — earning the word "conformant".

SAFE-TO-DELEGATE CERTIFIES A SPEC · CONFORMANCE-CHECK CERTIFIES AN EXECUTOR

They are duals. "Any conformant engine can pick it up" is **testable, not marketing** — a standard wins on its conformance suite and adapters, not its prose. `conformance-check.sh` certifies an executor at three levels; `ref-executor.sh` is the ~60-line worked example of the consumer contract.

LEVEL	THE EXECUTOR MUST...	WHAT IT BUYS
L0	Read the format + run the evals — make the Exit Check return 0 on a solvable spec.	Safe for supervised, one-shot dispatch.
L1	L0 + honor the lifecycle — acquire the lock (status → in-progress), release on success (→ done).	Safe for a shared backlog; no double-picking.
L2	L1 + honor the budget — on an unsolvable spec, stop and park with a reason instead of looping forever.	Safe for unattended operation; runaway tasks self-terminate.

Status lifecycle ↔ A2A interop

The status transitions map onto the Linux Foundation A2A `TaskState` enum, so any A2A-aware dispatcher can orchestrate Task-Specs:

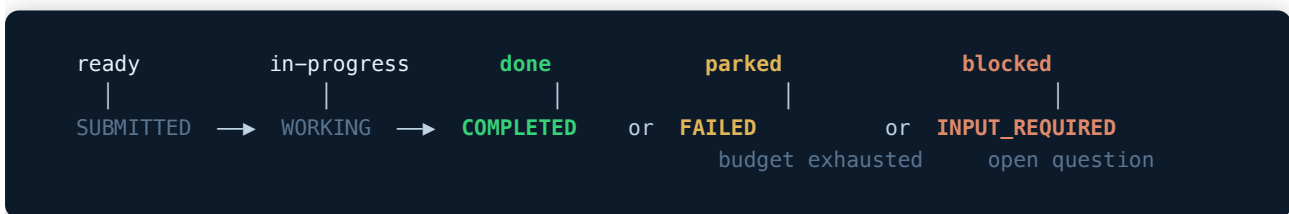


FIG 4 Task-Spec statuses ↔ the A2A TaskState enum — parked is a first-class honest failure (budget exhausted, with a reason), blocked is a first-class request for human input.

10 Backlog architecture — five layers of state.

ATOMIC · LOGGED · GIT-COMMITTED · CRASH-RECOVERABLE BY DESIGN

LAYER	ROLE
1 · <code>git</code>	The permanent record. Every transition is a commit; history is the audit trail of last resort.
2 · <code>tasks/</code> folder	The backlog itself: active specs at the root, <code>done/</code> and <code>parked/</code> subdirs via <code>archive.sh</code> , optional <code>queue/</code> for batch output.
3 · <code>frontmatter status</code>	The authoritative per-file truth. Changed only through <code>transition-status.sh</code> (lock + ledger) — hand-editing bypasses both.
4 · <code>_state.yaml</code>	Derived index for fast queries. Rebuildable at any time with <code>rebuild-state.sh</code> ; a pre-commit hook catches drift from direct writes.
5 · <code>_metrics.jsonl</code>	Append-only forensic ledger — one JSON line per event, queried by <code>query-metrics.sh</code> . Never rewritten, only appended.

THE INVARIANT

Successful validation idempotently snapshots frontmatter into `_state.yaml`; failed validation writes nothing — broken specs never pollute the audit trail. Frontmatter is authoritative; everything derived can be rebuilt.

THE THESIS, IN ONE LINE

"Specs that verify themselves keep humans at the edges of the loop — at intent-setting and acceptance review — not in the middle of every iteration." The mission is to be the open, atomic, vendor-neutral, self-verifying unit-of-work format any conformant agentic system can produce, consume, and trust.

Source: `skills/task-spec · SKILL.md v3.2.0` · design system: the Converge canonical blueprint · `task-spec` is Pass 5B — Fork B's cornerstone unit